项目计划书

项目名称： 校园宿舍报修与工单管理系统
学 院： 创业学院
小组序号： 待填写
成员姓名： 邓杰 、尹添一、陈梓轩、胡宇翔
指导老师： 尹兆远
提交日期： 2026年4月20日

一、项目概述

1. 项目背景

高校宿舍中常见灯具损坏、水电故障、门窗损坏、网络问题等报修事项。传统处理方式通常依赖电话、微信群或纸质登记，存在信息记录不完整、处理流程不透明、责任划分不清、统计分析困难等问题。为提高宿舍后勤服务效率，提升报修处理的规范化和信息化水平，本项目拟开发“校园宿舍报修与工单管理系统”，实现报修申请、工单流转、维修处理、评价反馈和数据统计等功能，形成一个可运行、可演示的完整信息系统。

2. 项目目标

本项目拟实现以下目标：

1. 为学生提供便捷的线上报修入口，支持提交报修申请、上传故障图片、查看工单进度。
2. 为管理员提供工单审核、分派、状态跟踪、用户管理和统计分析功能。
3. 为维修人员提供接单、处理、维修记录填写和结果反馈功能。
4. 构建一个前后端分离、结构清晰、便于团队协作开发和课程演示的信息系统。

3. 开发环境与条件

本项目采用前后端分离开发模式，开发环境与条件如下：

- **前端开发工具：** VS Code
- **后端开发工具：** IntelliJ IDEA
- **数据库工具：**DataGrip
- **接口测试工具：** Postman
- **版本管理工具：** Git + GitHub
- **运行环境：** Windows
- **部署方式：** 开发阶段本地部署，演示阶段可采用本地运行或云服务器部署
- **数据库：** MySQL 8.0

二、需求分析

1. 目标用户与使用场景

本系统主要面向以下三类用户：

（1）学生用户
学生可登录系统后提交宿舍报修申请，填写故障类型、宿舍信息、问题描述并上传图片；提交后可随时查看工单处理进度，在维修完成后确认结果并进行评价。

（2）管理员用户
管理员负责审核报修信息、分派维修人员、监督工单状态流转、维护故障分类与用户信息，并查看工单数量、完成率、故障类型分布等统计数据。

（3）维修人员用户
维修人员可查看待处理工单、选择接单、更新维修状态、填写维修记录并上传维修结果图片。

典型使用场景包括：学生发现宿舍设施损坏后在线提交报修；管理员审核工单后分配维修人员；维修人员上门维修并提交处理记录；学生确认完成并评价；管理员按时间维度查看统计报表。

2. 核心功能清单

根据项目需求，系统拟实现以下核心功能：

（1）学生端功能

- 用户登录
- 宿舍信息绑定
- 提交报修申请
- 上传故障图片
- 查看我的工单
- 查看工单处理进度
- 确认维修完成
- 提交评价与反馈

（2）管理端功能

- 工单列表管理
- 工单审核
- 工单分派
- 工单状态跟踪
- 用户管理
- 故障分类管理
- 数据统计分析

（3）维修端功能

- 待处理工单查看
- 接单/拒单
- 更新维修状态
- 填写维修记录
- 上传维修结果图片

（4）系统公共功能

- 角色登录认证
- 角色权限控制
- 统一接口返回
- 异常处理与提示
- 图片文件管理

3. 非功能需求

- **性能要求：** 支持几十到上百用户并发访问，普通查询响应时间控制在 2 秒内。
- **安全要求：** 实现登录认证、角色权限控制、密码加密存储、敏感接口访问限制。
- **兼容性要求：** 前端页面兼容主流现代浏览器，系统可在 Windows 环境下部署运行。
- **可维护性要求：** 采用前后端分离架构，便于多人分工协作和后续功能扩展。
- **可演示性要求：** 界面简洁、流程完整，满足课程汇报与系统展示需求。

三、技术路线

1. 总体架构

本系统拟采用**前后端分离的 B/S 架构**。前端主要负责页面展示、表单交互、状态可视化和图表展示；后端负责业务逻辑处理、接口提供、登录认证、权限控制和数据访问；数据库负责用户、工单、维修记录、评价等核心业务数据存储。

系统总体技术架构为：

Vue 3 前端 + Spring Boot 后端 + MySQL 数据库

该方案符合课程允许的技术范围，开发资料丰富，团队协作成本较低，适合在课程周期内完成一个完整、可运行、可演示的信息系统。

2. 主要技术

（1）前端技术

- Vue 3
- Vite
- Vue Router
- Pinia
- Element Plus
- Axios
- ECharts

选型说明：
Vue 3 生态成熟，适合快速开发课程项目；Element Plus 组件丰富，便于搭建后台管理系统页面；Pinia 适合状态管理；Axios 便于完成前后端接口联调；ECharts 可用于实现故障类型占比、工单趋势、人员处理量等图表展示。

（2）后端技术

- Spring Boot 3
- MyBatis-Plus
- JWT
- Lombok
- Maven

选型说明：
Spring Boot 适合快速构建后端服务；MyBatis-Plus 可降低基础 CRUD 开发成本；JWT 用于实现登录认证和角色访问控制；Lombok 能简化实体类与基础代码；Maven 用于项目依赖管理和构建。该组合兼顾开发效率、系统规范性与课程项目的可实现性。

（3）数据库与文件存储

- MySQL 8.0
- 本地目录文件存储（用于故障图片与维修结果图片）
- Redis 作为可选优化组件，不纳入首版必做范围

选型说明：
MySQL 部署简单、资料丰富、与 Spring Boot + MyBatis-Plus 组合成熟，适合作为本项目的主要数据库。图片上传部分采用本地目录存储，数据库中保存图片路径，以降低开发复杂度并优先保证主流程落地。

3. 数据设计初步考虑

（1）主要实体或对象

系统初步涉及以下核心实体：

- 用户（User）
- 报修工单（RepairOrder）
- 故障分类（RepairCategory）
- 维修记录（RepairRecord）
- 评价反馈（Evaluation）
- 公告信息（Notice，可选）

（2）关键数据表或数据关系

1. 用户表（user）
主要字段：id、username、password、real_name、role、phone、dorm_building、dorm_room、status、create_time

2. 报修工单表（repair_order）
主要字段：id、order_no、user_id、title、category_id、description、image_url、dorm_building、dorm_room、status、priority、assigned_worker_id、submit_time、finish_time

3. 故障分类表（repair_category）
主要字段：id、category_name、description

4. 维修记录表（repair_record）
主要字段：id、order_id、worker_id、action_desc、action_time、result_image、status_after_action

5. 评价表（evaluation）
主要字段：id、order_id、user_id、score、content、create_time

6. 公告表（notice，可选）
主要字段：id、title、content、publish_time、publisher_id

核心数据关系如下：

- 一个学生用户可以提交多个报修工单；
- 一个工单属于一个故障分类；
- 一个工单可以对应多条维修记录；
- 一个工单最终可对应一条评价记录；
- 一个维修人员可以处理多个工单。

四、进度安排

1. 阶段划分

根据课程安排，项目计划书需在 **4 月 24 日前**提交，中期报告需在 **5 月 6 日前**提交，软件说明书需在 **5 月 13 日前**提交。结合课程节奏，本项目拟划分为以下四个阶段：

第一阶段：需求分析与总体设计（现在—4月24日前）
完成选题确认、需求分析、技术方案确定、项目计划书撰写，完成系统模块划分、业务流程梳理、数据库初步设计，并建立远端 Git 仓库和项目基础结构。

第二阶段：核心功能开发（4月25日—5月5日）
完成登录认证、学生报修、管理员审核与分派、维修人员接单与处理、数据库建表与基础接口联调，实现系统主流程。

第三阶段：中期整理与测试（5月6日前）
整理中期报告，补充系统架构设计、模块说明、E-R 图、关键表设计、页面原型与业务流程；同时开展基础测试并修复主要问题。

第四阶段：系统完善与结项（5月7日—5月13日前）
补充统计图表、优化页面交互、完善异常处理、整理软件说明书、准备最终汇报材料与系统演示。

2. 时间进度表

阶段	时间安排	主要任务	预期成果
第一阶段	4月20日—4月24日	需求分析、技术方案确定、数据库初步设计、项目计划书编写、仓库初始化	项目计划书、模块设计说明、初步表
第二阶段	4月25日—5月5日	前后端项目搭建、核心页面与接口开发、工单主流程实现、联调测试	可运行的系统初版、核心功能代码、
第三阶段	5月1日—5月6日	中期文档整理、E-R 图完善、测试与 Bug 修复	中期报告、E-R 图、测试记录、问题
第四阶段	5月7日—5月13日	图表统计、页面优化、说明书整理、汇报材料准备	软件说明书、演示 PPT、最终代码与

上述安排既对应课程规定的提交节点，也便于保证每位成员在不同阶段形成可追踪的 Git 留痕成果。

3. 风险与应对

（1）开发周期较短，功能无法全部完成
应对思路：优先保证系统主流程，即“登录—报修—审核—分派—处理—评价”完整可运行；附加功能如公告、消息提醒、Redis 优化等列为扩展项。

（2）角色权限设计复杂，容易出现越权问题
应对思路：前期先明确学生、管理员、维修人员三类角色的菜单权限与接口权限，统一设计权限表和状态流转规则。

（3）前后端联调时接口字段不一致
应对思路：在开发早期先约定接口命名、请求参数和统一返回格式，使用 Postman 进行接口测试。

（4）多人协作导致 Git 冲突或提交证据不足
应对思路：按模块分支开发，定期合并；所有过程文档、测试记录、会议纪要和设计文件均以 Markdown 或版本化文件形式纳入仓库，确保个人贡献可追踪。课程考核明确说明：远端 Git 内容为主要评分依据，非编码工作也必须形成仓库中的版本化成果。

五、成员分工

1. 成员信息

成员	班级	学号	Git账号	主要任务
邓杰	24级计算机科学与技术创业菁英班	202405550309	dengjie86	项目负责人、后端主程；负责系统架构、登录认证、权限控制、
尹添一	24级计算机科学与技术创业菁英班	202405550301	gubei026314	前端主程；负责登录页、学生端、管理端页面开发，前后端接口
陈梓轩	24级计算机科学与技术创业菁英班	202405550319	yihai23	数据库与后端协助；负责数据库设计、实体与 Mapper、统计分
胡宇翔	24级计算机科学与技术创业菁英班	202405550317	saintwugo	测试与文档负责人；负责测试用例、测试记录、计划书、中期报

上述分工符合课程对“每位成员需明确主要负责部分、最终提交产物、对应仓库目录或文档类型”的要求。

2. Git协作方式

本项目采用远端 Git 仓库进行协作管理，源码与过程文档均以仓库内容为准，不单独提交源码压缩包。协作方式如下：

1. 建立统一远端仓库，主分支为 `main` ，各成员按模块建立个人或功能分支进行开发。
2. 前端、后端、数据库、测试与文档分别维护相对独立的目录，减少多人同时修改同一文件的冲突。
3. 合并代码前先同步主分支内容，完成自测后再提交 pull request 或由组长统一合并。
4. 所有成员需保持持续、真实的提交记录，提交内容须与本人分工一致。
5. 除代码外，需求文档、数据库设计、测试用例、会议纪要、任务分解表、进度跟踪表、阶段汇报材料等均纳入版本管理，用于支撑个人成绩认定。

3. 阶段性成果计划

第一阶段成果：

- 项目计划书
- 需求分析文档
- 系统模块划分说明
- 数据库初步设计
- Git 仓库初始化与目录规范

第二阶段成果：

- 前后端项目基础框架
- 登录模块
- 学生报修模块
- 管理员审核分派模块
- 维修人员处理模块
- 基础接口文档与联调记录

第三阶段成果：

- 中期报告
- E-R 图与表结构说明
- 页面原型或功能截图
- 测试用例与测试记录
- Bug 修复记录

第四阶段成果：

- 数据统计图表模块
- 软件说明书
- 演示 PPT
- 最终系统部署与演示材料

以上成果安排与课程评分中“项目计划书—中期设计文档—最终系统完成度—软件说明书与最终汇报”的考核结构保持一致。

附录

1. **远端Git仓库信息：** https://github.com/dengjie86/information-systems-practice
2. **其他说明：**
   - 本项目开发过程文档优先使用 Markdown 格式撰写，图片单独存放在仓库目录中，便于 Git 追踪版本变化。
   - 项目优先保证主流程完整实现，在时间允许的情况下增加统计图表、公告管理、消息提醒等扩展功能。
   - 软件说明书阶段将基于 Git 仓库中的过程文档进一步整理为正式文档。